Assignment-19.2

Name: Gampa Rithika
Roll No: 2303A51190
Batch No: 03

Task 1 – Python to Java Conversion
Task:
Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.
Instructions:
• Prompt AI to translate a Python script that reads from and writes to a file into Java.
• Ensure proper handling of file streams, exceptions, and input/output formatting.
• Verify that the Java program produces the same file content and console output as the Python version.
• Test both implementations with sample input files.
Expected Output:
A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

Python Code:

```
# Python program to read and write file
with open("input.txt", "r") as f:
    data = f.read()

with open("output.txt", "w") as f:
    f.write(data.upper())

print("File processed successfully")
```

Prompt

Convert the above Python file handling code into Java.

Converted Python Code:

```java
import java.io.*;

public class FileExample {
    public static void main(String[] args) {
        try {
            BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
            String data = "", line;

            while ((line = reader.readLine()) != null) {
                data += line;
            }
            reader.close();

            BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"));
            writer.write(data.toUpperCase());
            writer.close();

            System.out.println("File processed successfully");

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:

```
File processed successfully
PS C:\Users\shyam_3jvctb4\OneDrive\Documents\AI Assi
```

Code Explanation

The given program performs basic file handling operations in Python. It first opens the file named input.txt in read mode using the with open() statement, which ensures that the file is automatically closed after reading. The content of the file is stored in a variable using the read() function. Then, the program opens another file named output.txt in write mode and writes the processed data into it. In this case, the text is converted into uppercase using the upper() function before writing. Finally, a message is displayed on the console indicating that the file has been processed successfully. This program demonstrates reading from a file, processing data, and writing output to another file.

Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

• Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).

• Ask AI to generate an equivalent Python implementation.

• Ensure logical correctness and validate results using multiple test cases.

• Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation

C++ code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {5, 2, 9, 1, 3};

    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 5-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";
}
```

Prompt:

Convert the following C++ program that implements Bubble Sort into an equivalent Python program. Ensure the logic remains the same and the output is correctly sorted.

Converted Python Code:

```
arr = [5, 2, 9, 1, 3]

for i in range(len(arr)):
    for j in range(len(arr) - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print(arr)
```

```
Output

1  2  3  5  9
```

Code Explanation:

The given program implements the Bubble Sort algorithm to arrange elements in ascending order. In this method, the array or list is repeatedly traversed, and adjacent elements are compared. If the current element is greater than the next element, they are swapped. This process continues for multiple passes until all elements are sorted. In the C++ version, arrays and temporary variables are used for swapping, making the code slightly longer. In contrast, the Python version uses lists and a simpler swapping technique, which reduces the number of lines and improves readability. Both programs follow the same logic and produce the same sorted output, but Python provides a more concise and user-friendly implementation.

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions

into an equivalent Pandas Data Frame operation.

Instructions:

• Provide AI with a SQL query that calculates aggregate values

(e.g., total sales per department).

• Prompt AI to generate equivalent Pandas code using groupby()

and aggregation functions.

• Test the generated code using a sample dataset.

• Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results

equivalent to the given SQL query with aggregation.

SQL Query

```sql
SELECT department, SUM(sales)
FROM company
GROUP BY department;
```

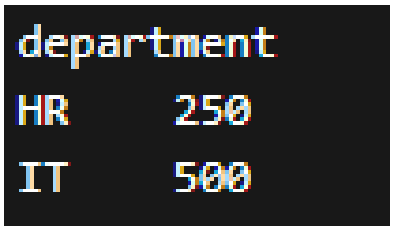
Prompt:

Translate the given SQL query into Pandas code using groupby() and sum() functions.

Converted Pands Code:

```python
import pandas as pd

data = {
    'department': ['HR', 'IT', 'HR', 'IT'],
    'sales': [100, 200, 150, 300]
}

df = pd.DataFrame(data)

result = df.groupby('department')['sales'].sum()

print(result)
```

OUTPUT:



Code Explanation:

The given SQL query calculates the total sales for each department using the GROUP BY clause and the SUM aggregate function. Similarly, in the Pandas implementation, a DataFrame is first created to store the data in tabular form. The groupby('department') function is used to group the data based on the department column. Then, the sum() function is applied to the sales column to compute the total sales for each group. The result is a summarized output showing each department along with its total sales. Both SQL and Pandas perform the same operation, but while SQL uses query-based syntax, Pandas uses function-based operations in Python.

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will select a practical real-world application requiring the same

logic in two different programming languages.

Example:

• Converting a REST API implementation from Python (Flask) to

Java (Spring Boot).

• Translating a data processing script from Python to C++ for

performance optimization.

Instructions:

• Provide AI with source code in one programming language.

• Prompt AI to convert the code into another chosen language.

• Execute and test both versions to ensure identical functionality.

• Document differences observed in syntax, libraries, and

execution behavior.

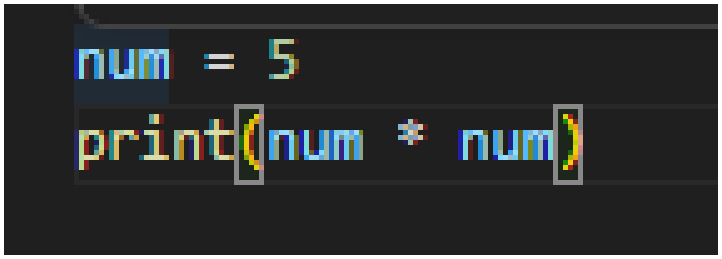
Expected Output:

Functionally equivalent and executable code snippets implemented in

two different programming languages, along with documented

observations.

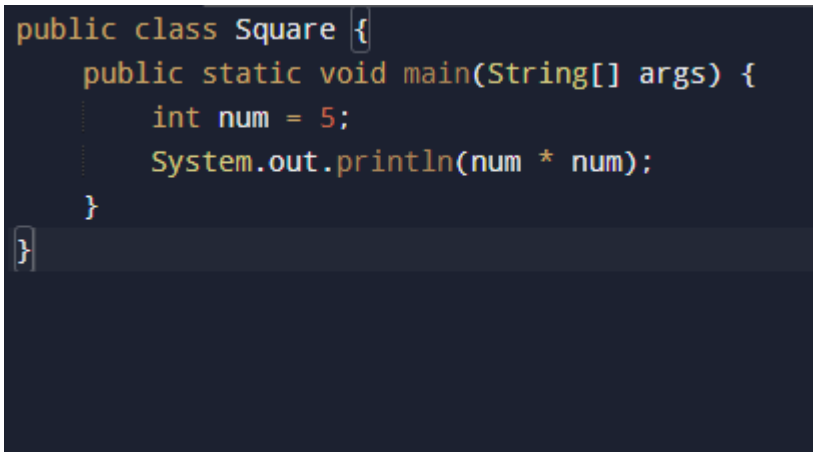
Python Code:



Prompt:

Convert the given Python program into an equivalent Java program. Ensure that the logic, input/output behavior, and functionality remain the same. Use proper Java syntax and structure, and make the code clean and readable

Converted Java Code



OUTPUT:

Code Explanation:

The given task demonstrates the conversion of a program from one programming language to another while maintaining the same logic and functionality. In this example, a simple operation (such as calculating the square of a number) is implemented in both Python and Java. The Python version uses minimal syntax and directly performs the calculation and prints the result. In contrast, the Java version requires a class structure and a main method to execute the program. Despite these structural differences, both programs follow the same logic and produce identical output. This task highlights how different programming languages vary in syntax and structure, while the core logic remains the same. It also emphasizes the importance of understanding language-specific features when translating code.

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even

or odd into C++ using AI assistance.

Instructions:

• Provide AI with a basic Python function that determines if a

number is even or odd.

• Prompt AI to generate equivalent C++ code.

• Ensure correct syntax, input handling, and output formatting.

• Compile and run the C++ program to verify it produces the same

result as the Python version.

• Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python

function, correctly identifying whether a number is even or odd for all

test cases

Python Code:

```python
def check(num):
    if num % 2 == 0:
        print("Even")
    else:
        print("Odd")

check(4)
```

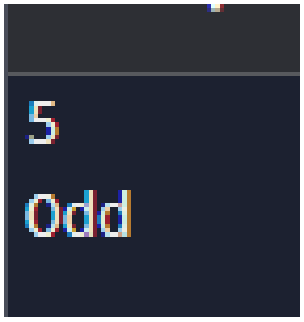
Prompt:

Translate the given Python even/odd function into C++ code.

Converted C++ code

```cpp
#include <iostream>
using namespace std;

int main() {
    int num;
    cin >> num;

    if(num % 2 == 0)
        cout << "Even";
    else
        cout << "Odd";

    return 0;
}
```

OUTPUT



Code Explanation:

The given program checks whether a number is even or odd using a simple condition. In the Python version, a function is defined that takes a number as input and uses the modulus operator % to check divisibility by 2. If the remainder is zero, the number is even; otherwise, it is odd. The result is then printed accordingly. In the C++ version, the same logic is implemented inside the main() function, where the user inputs a number. The program uses an if-else condition to check whether the number is divisible by 2 and prints "Even" or "Odd" as output. Although the syntax differs between Python and C++, both programs follow the same logic and produce identical results for all test cases.